Technical Implementation Brief — Policy Pilot

AI-assisted, human-in-the-loop access-review system. Every claim below cites the exact file that implements it.

---

1. Architecture Overview

Policy Pilot separates ingestion, AI recommendation, and human execution into distinct trust boundaries. A NestJS API (`backend/src/ingestion/`) validates and enqueues access-request webhooks, returning 202 Accepted immediately. A BullMQ worker (`backend/src/queue/access-request.processor.ts`) performs entitlement lookup, RAG retrieval, and LLM recommendation asynchronously. A React + TanStack Query dashboard (`frontend/src/`) is the only interface a human reviewer uses to Approve, Deny, or Override; only that decision can reach the mocked downstream execution adapter (`backend/src/decisions/execution-adapter.service.ts`).

2. PDF Parsing Strategy

Native PDF policies are parsed with **pdf-parse**, then normalized: numbered headings (e.g. “2.1…”) are promoted to Markdown headings by **pdfTextToMarkdown** (`backend/src/rag/pdf.ts`), so PDF and Markdown policy documents flow through one identical chunking path rather than two parallel implementations.

3. Chunking Strategy and Trade-offs

Chunking splits on Markdown headings first (`backend/src/rag/chunking.ts`); a section is only subdivided into 500-token windows with a 50-token (~10%) overlap if it exceeds the token budget. **Trade-off:** heading-first splitting keeps each policy rule semantically whole and individually citable — a naive fixed-size window would routinely sever a rule’s condition from its consequence — at the cost of variable chunk sizes. Full rationale in `docs/CHUNKING_STRATEGY.md`.

4. Metadata-Tagging Approach

Each chunk is tagged with document_name, version, effective_date, section, policy_domain, department, cost_center, target_system, and entitlement_type (`backend/src/rag/metadata.ts`), extracted from document headers and regex matches against known code lists — enabling metadata pre-filtering at query time rather than relying on embedding similarity alone.

5. Retrieval and Filtering Strategy

Chunks are embedded with OpenAI **text-embedding-3-small** (1536-dim) and stored in pgvector with an HNSW index (cosine distance, `backend/prisma/schema.prisma`). `rag.service.ts` embeds the incoming query, runs a cosine-similarity search, and applies optional metadata filters (policy domain, cost center, target system) as parameterized SQL before returning the top-k chunks as citation candidates.

6. Prompt and Structured-Output Design

Prompts are version-controlled files (`backend/src/agent/prompts/v1|v2|v3.prompt.ts`, each with an in-file changelog); v3 is active. The untrusted requester justification is fenced between explicit delimiters, and the system prompt instructs the model never to follow instructions found inside it. The LLM response is validated against a Zod schema (`agent.types.ts`) before persistence; malformed output is retried up to 3 attempts, then the request escalates to human review.

7. Idempotency Design

A dual gate — an Idempotency-Key header or the request’s own request_id (`backend/src/ingestion/idempotency.service.ts`) — is backed by database-level unique constraints on both columns, so duplicate execution is impossible even under a race. BullMQ’s jobId is the stable accessRequestId, so a requeue (e.g. after a crash) never creates a duplicate job.

8. Append-Only Audit Strategy

audit_log is append-only **structurally**, not by convention: Postgres BEFORE UPDATE / BEFORE DELETE triggers (FOR EACH ROW) raise an exception on any mutation attempt (`migrations/20260722015659_core_tables/migration.sql`). AiRecommendation and HumanDecision are separate tables, so the AI’s suggestion and the human’s final decision are always independently reconstructable.

9. Downstream Rate-Limit Handling

The BullMQ worker is configured with a limiter of **{ max: 60, duration: 60_000 }** (`backend/src/queue/queue.module.ts`), matching the assignment’s assumed 60/min downstream capacity; a burst of up to 300/min is absorbed by the queue rather than overwhelming the (mocked) downstream system.

10. Retry and Backpressure Strategy

Jobs retry up to 3 times with exponential backoff and jitter (1000ms base delay, 0.5 jitter). There is no separate Redis dead-letter queue: a FAILED row in Postgres plus its audit trail is the durable failure record, with a CLI replay tool (**queue:replay-failed**) and a `StaleRequestSweeperService` that recovers requests orphaned by a crash between the DB commit and the original enqueue call.

11. PII-Masking Approach

employee_id and cost_center are partially masked (e.g. **EMP-***90**) before any log line or audit payload is written (`backend/src/common/pii.util.ts`); free-text fields and error stack traces are routed through the same redactor before being logged.

12. Evaluation and Observability Strategy

A version-controlled golden dataset (`backend/golden-dataset.json`, 8 cases covering approval, denial, escalation, SoD conflict, insufficient evidence, duplicate entitlement, invalid input, and prompt injection) is scored by `backend/src/eval/run-eval.ts` for schema validity, retrieval quality, grounding, decision correctness, citation quality, latency (p50/p95), and estimated cost. Observability today is structured NestJS Logger output (including per-call LLM latency/cost) plus /health and /health/ready endpoints — there is no OpenTelemetry tracing or metrics exporter yet.

13. Major Assumptions and Trade-offs

Separation-of-duties conflicts are evaluated by deterministic code (`backend/src/entitlements/sod-conflict-pairs.constant.ts`), not LLM inference — a detected conflict short-circuits straight to DENY with zero LLM cost, zero hallucination risk, and no policy citations, since the fact is already fully determined by the entitlement registry. The downstream execution adapter only logs its action rather than mutating a real system, and reviewer authentication is a mocked identity picker rather than real SSO — both explicitly scoped as out of bounds for this assignment. **Known gaps:** no explicit LLM call timeout, no frontend test suite, and no committed evaluation report with real run numbers yet.